

Code source

Druid's Adventure

Auteur : Valentin Tercier

Projet : Travail Pratique Individuel

Technologie : Unity / C#

Date : 07.05.2026

Table des matières du code source

- [EcranDeFin](#)
- [GameManager](#)
- [MenuAccueil](#)
- [MenuChoixNiveau](#)
- [PlayerStatManager](#)
- [NiveaTroisManager](#)
- [FollowCamera](#)
- [AnimationState](#)
- [Bear](#)
- [CanAttack](#)
- [DetectionSolDroite](#)
- [DetectionSolGauche](#)
- [Bird](#)
- [DetectionInteraction](#)
- [DetectionSol](#)
- [Player](#)
- [Fish](#)
- [Entity](#)
- [GestionPersonnage](#)
- [Water](#)
- [DetectorPlayer](#)
- [Sanglier](#)
- [Abeille](#)
- [DetectorEnnemiSol](#)
- [DetectorObstacle](#)
- [DepotCrystal](#)
- [FinirLeNiveau](#)
- [Brick](#)
- [SpikeAnimationState](#)
- [spikeGroundScript](#)

\newpage

EcranDeFin

Chemin : C:/Users/Admin/Druid's Adventure/Assets/GameManager/EcranDeFin.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour changer de scène lorsque le joueur appuie le bouton
continuer
using UnityEngine;
using UnityEngine.SceneManagement;

public class EcranDeFin : MonoBehaviour
{
    // Start is called once before the first execution of Update after the
    MonoBehaviour is created
    /// <summary>
    /// fonction pour changer de scène
    /// </summary>
    public void continuer()
    {
        SceneManager.LoadScene("MenuChoixNiveau");
    }
}
```

GameManager

Chemin : C:/Users/Admin/Druid's Adventure/Assets/GameManager/GameManager.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour gérer l'écran de fin en fin de partie il va permettre
d'afficher les stats et si le joueur est mort ou pas
using TMPro;
using UnityEngine;

public class GameManager : MonoBehaviour
{
    /// <summary>
    /// Nombre d'orbes de mana récupérés pendant le niveau.
    /// </summary>
    int _nbrOrbreManaRecuperer;

    /// <summary>
    /// Nombre d'orbes de vie récupérés pendant le niveau.
    /// </summary>
```

```
int _nbrOrbreVieRecuperer;

/// <summary>
/// Portail permettant de terminer le niveau.
/// </summary>
public GameObject portail;

/// <summary>
/// Référence au script de fin de niveau.
/// </summary>
FinirLeNiveau _finirLeNiveau;

//Canvas

/// <summary>
/// Élément UI affichant la vie.
/// </summary>
public GameObject vie;

/// <summary>
/// Élément UI affichant la mana.
/// </summary>
public GameObject mana;

/// <summary>
/// Image principale de l'interface.
/// </summary>
public GameObject image;

// écran de fin

/// <summary>
/// Image affichée à la fin du niveau.
/// </summary>
public GameObject imageFin;

/// <summary>
/// Texte affiché à la fin du niveau.
/// </summary>
public GameObject TextFin;

/// <summary>
/// Bouton permettant de continuer après la fin du niveau.
/// </summary>
public GameObject btnContinuer;

/// <summary>
/// Texte affichant le nombre d'orbes de vie récupérés.
/// </summary>
public GameObject statVie;

/// <summary>
/// Texte affichant le nombre d'orbes de mana récupérés.
/// </summary>
```

```
public GameObject statMana;

/// <summary>
/// Composant TextMeshPro du texte des orbes de vie.
/// </summary>
TextMeshProUGUI _statVieText;

/// <summary>
/// Composant TextMeshPro du texte des orbes de mana.
/// </summary>
TextMeshProUGUI _statManaText;

/// <summary>
/// GameObject du joueur.
/// </summary>
public GameObject player;

private GestionPersonnage gp;


/// <summary>
/// Initialisation des composants nécessaires.
/// </summary>
void Start()
{
    _finirLeNiveau = portail.GetComponent<FinirLeNiveau>();
    _statVieText = statVie.GetComponent<TextMeshProUGUI>();
    _statManaText = statMana.GetComponent<TextMeshProUGUI>();
    gp = player.GetComponent<GestionPersonnage>();
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Vérifie si le niveau est terminé.
/// </summary>
void Update()
{
    // Si le niveau est terminé, affiche l'écran de fin
    if (_finirLeNiveau.IsFinished())
    {
        vie.SetActive(false);
        mana.SetActive(false);
        image.SetActive(false);
        imageFin.SetActive(true);
        TextFin.SetActive(true);
        btnContinuer.SetActive(true);

        _statVieText.text =
            "Nombre d'orbre vie rammasser : " +
            _nbrOrbreVieRecuperer;

        _statManaText.text =
```

```
        "Nombre d'obre mana rammasser : " +
        _nbrOrbreManaRecuperer;

        player.SetActive(false);
    }
    if (gp.IsDead())
    {
        vie.SetActive(false);
        mana.SetActive(false);
        image.SetActive(false);
        imageFin.SetActive(true);
        TextFin.SetActive(true);
        btnContinuer.SetActive(true);

        TextFin.GetComponent<TextMeshProUGUI>().text = "";

        _statVieText.text = "Vous êtes mort ";

        _statManaText.text = "";
        Debug.Log("mort");
    }
}

/// <summary>
/// Ajoute une orbe de mana au compteur.
/// </summary>
public void AjoutOrbreMana()
{
    _nbrOrbreManaRecuperer++;
}

/// <summary>
/// Ajoute une orbe de vie au compteur.
/// </summary>
public void AjoutOrbreVie()
{
    _nbrOrbreVieRecuperer++;
}

/// <summary>
/// Retourne le nombre d'orbes de vie récupérés.
/// </summary>
/// <returns>
/// Nombre d'orbes de vie.
/// </returns>
public int OrbreVie()
{
    return _nbrOrbreVieRecuperer;
}

/// <summary>
/// Retourne le nombre d'orbes de mana récupérés.
/// </summary>
/// <returns>
```

```
    /// Nombre d'orbes de mana.  
    /// </returns>  
    public int OrbreMana()  
    {  
        return _nbrOrbreManaRecuperer;  
    }  
}
```

MenuAccueil

Chemin : C:/Users/Admin/Druid's Adventure/Assets/GameManager/MenuAccueil.cs

```
using TMPro;  
using UnityEngine;  
using UnityEngine.SceneManagement;  
  
public class MenuAccueil : MonoBehaviour  
{  
    //Valentin Tercier  
    //07.05.2026  
    //Ce script est utilisé pour gérer le menu d'accueil  
    public GameObject vie;  
    public GameObject mana;  
  
    void Start()  
    {  
        TextMeshProUGUI txt = vie.GetComponent<TextMeshProUGUI>();  
        txt.text = "Vie max = " + PlayerStatManager.Instance.maxHealth;  
        txt = mana.GetComponent<TextMeshProUGUI>();  
        txt.text = "Mana max = " + PlayerStatManager.Instance.maxMana;  
    }  
    public void Jouer()  
    {  
        SceneManager.LoadScene("MenuChoixNiveau");  
    }  
  
    public void Quitter()  
    {  
        Application.Quit();  
    }  
    // Update is called once per frame  
}
```

MenuChoixNiveau

Chemin : C:/Users/Admin/Druid's Adventure/Assets/GameManager/MenuChoixNiveau.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé menu des choix de niveau
using UnityEngine;
using UnityEngine.SceneManagement;

public class MenuChoixNiveau : MonoBehaviour
{
    // Start is called once before the first execution of Update after the
    MonoBehaviour is created
    public void NiveauUn()
    {
        SceneManager.LoadScene("NiveauUn");
    }

    public void NiveauDeux()
    {
        SceneManager.LoadScene("NiveauDeux");
    }

    public void NiveauTrois()
    {
        SceneManager.LoadScene("NiveauTrois");
    }

    public void Retour()
    {
        SceneManager.LoadScene("MenuAccueil");
    }
}
```

PlayerStatManager

Chemin : C:/Users/Admin/Druid's Adventure/Assets/GameManager/PlayerStatManager.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé lors du lancement du jeu pour sauvegarder le projet
using System.IO;

using UnityEngine;
```

```
public class PlayerStatManager : MonoBehaviour
{
    public static PlayerStatManager Instance;

    public int maxHealth;
    public int health;

    public int maxMana;
    public int mana;
    public bool HasCrystal;
    string filePath;

    private void Awake() {
        if(Instance != null)
        {
            Destroy(gameObject);
            return;
        }
        filePath = Application.persistentDataPath + "/StatData.json";
        Debug.Log(filePath);
        if (System.IO.File.Exists(filePath))
        {
            string donnee = File.ReadAllText(filePath);
            StatData stats = JsonUtility.FromJson<StatData>(donnee);
            health = stats.health;
            maxHealth = stats.healthMax;
            maxMana = stats.manaMax;
            mana = stats.mana;
        }
        else
        {
            health = 100;
            maxHealth = 100;
            maxMana = 60;
            mana = 60;
        }

        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    public void SaveStats(int currentHealth, int currentMana,int maxHealth,int
manaMax)
    {
        health = currentHealth;
        mana = currentMana;
        this.maxHealth = maxHealth;
        maxMana = manaMax;
    }

    public void SauvegarderDonnee(int healthMax,int health,int manaMax,int mana)
    {
        StatData stats = new StatData();
```



```

        stats.healthMax = healthMax;
        stats.health = health;
        stats manaMax = manaMax;
        stats.mana = mana;

        string donneeVieEtMana = JsonUtility.ToJson(stats);
        Debug.Log(filePath);
        Debug.Log(donneeVieEtMana);
        System.IO.File.WriteAllText(filePath,donneeVieEtMana);
        Debug.Log("DonnéeSauvegardé");
    }

    [System.Serializable]
    public class StatData
    {
        public int healthMax;
        public int health;
        public int manaMax;
        public int mana;
    }
}

```

NiveaTroisManager

Chemin : C:/Users/Admin/Druid's Adventure/Assets/NiveauTroisManager/NiveaTroisManager.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour activer le portail du niveau 3
using UnityEngine;

public class NiveaTroisManager : MonoBehaviour
{
    public GameObject Levier;
    Levier levier;
    public GameObject DepotCrystal;
    DepotCrystal depotCrystal;

    public GameObject portail;
    FinirLeNiveau finirLeNiveau;
    // Start is called once before the first execution of Update after the
    MonoBehaviour is created
    void Start()
    {
        levier = Levier.GetComponent<Levier>();
    }
}

```

```

        depotCrystal = DepotCrystal.GetComponentInChildren<DepotCrystal>();
        finirLeNiveau = portail.GetComponent<FinirLeNiveau>();
    }

    // Update is called once per frame
    void Update()
    {
        if(levier.isActivated && depotCrystal.isActivated)
        {
            finirLeNiveau.canBeTake = true;
        }
    }
}

```

FollowCamera

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Camera/Script/FollowCamera.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour suivre le joueur avec la camera
using UnityEngine;

public class FollowCamera : MonoBehaviour
{
    /// <summary>
    /// Cible actuellement suivie par la caméra.
    /// </summary>
    private GameObject _target;

    /// <summary>
    /// GameObject contenant le script GestionPersonnage.
    /// </summary>
    public GameObject player;

    /// <summary>
    /// Référence au script GestionPersonnage.
    /// </summary>
    private GestionPersonnage _gestionPersonnage;

    /// <summary>
    /// Décalage appliqué sur l'axe Z de la caméra.
    /// </summary>
    public int offsetZ = 0;

    /// <summary>
    /// Initialisation des composants nécessaires.
    /// </summary>

```

```
void Start()
{
    _gestionPersonnage =
        player.GetComponent<GestionPersonnage>();
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Permet de suivre la forme actuellement contrôlée.
/// </summary>
void Update()
{
    // Vérifie si le joueur contrôle le druide
    if(_gestionPersonnage.currentPersonnage ==
        GestionPersonnage.TypePersonnage.Druide)
    {
        _target = _gestionPersonnage.DruideGo;
    }

    // Vérifie si le joueur contrôle l'ours
    else if(_gestionPersonnage.currentPersonnage ==
        GestionPersonnage.TypePersonnage.Bears)
    {
        _target = _gestionPersonnage.BearGo;
    }

    // Vérifie si le joueur contrôle le poisson
    else if(_gestionPersonnage.currentPersonnage ==
        GestionPersonnage.TypePersonnage.Fish)
    {
        _target = _gestionPersonnage.FishGo;
    }

    // Vérifie si le joueur contrôle l'oiseau
    else if(_gestionPersonnage.currentPersonnage ==
        GestionPersonnage.TypePersonnage.Bird)
    {
        _target = _gestionPersonnage.BirdGo;
    }

    // Vérifie si une cible existe
    if(_target == null)
    {
        return;
    }

    // Déplace la caméra sur la position de la cible
    transform.position =
        new Vector3(
            _target.transform.position.x,
            _target.transform.position.y,
            offsetZ
        );
}
```

```
}  
}
```

AnimationState

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Bear/Script/AnimationState.cs

```
//Valentin Tercier  
//07.05.2026  
//Ce script est utilisé fluidifier les attaques  
using UnityEngine;  
  
public class AnimationState : MonoBehaviour  
{  
    private Bear _bear;  
  
    void Start()  
    {  
        _bear = GetComponentInParent<Bear>();  
    }  
    void Attack()  
    {  
        _bear.EnleverDegat();  
    }  
  
    void Reset()  
    {  
        _bear.ResetAttack();  
    }  
}
```

Bear

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Bear/Script/Bear.cs

```
//Valentin Tercier  
//07.05.2026  
//Ce script est utilisé pour le mouvement de l'ours ces attaques etc...  
using UnityEngine;  
using UnityEngine.InputSystem;  
  
public class Bear : Entity  
{  
    /// <summary>
```

```
/// GameObject principal de l'ours.  
/// </summary>  
public GameObject bear;  
  
/// <summary>  
/// Direction horizontale du déplacement.  
/// </summary>  
float _directionX;  
  
/// <summary>  
/// Rigidbody2D utilisé pour gérer les déplacements et la physique.  
/// </summary>  
Rigidbody2D _rb;  
  
/// <summary>  
/// Vitesse de déplacement de l'ours.  
/// </summary>  
float _speed = 3f;  
  
/// <summary>  
/// Animator utilisé pour gérer les animations.  
/// </summary>  
Animator _animator;  
  
/// <summary>  
/// Détection du sol à droite du joueur.  
/// </summary>  
DetectionSolDroite _detectionSolDroite;  
  
/// <summary>  
/// Détection du sol à gauche du joueur.  
/// </summary>  
DetectionSolGauche _detectionSolGauche;  
  
/// <summary>  
/// Ennemi actuellement détecté pouvant recevoir des dégâts.  
/// </summary>  
Entity _entityEnemy;  
  
/// <summary>  
/// Vérifie si l'ours est actuellement en train d'attaquer.  
/// </summary>  
bool _isAttacking = false;  
  
/// <summary>  
/// Script permettant de détecter si une attaque peut toucher un ennemi.  
/// </summary>  
CanAttack _canAttack;  
  
/// <summary>  
/// Initialisation des composants nécessaires.  
/// </summary>  
void Start()  
{
```

```
base.Start();

_rb = bear.GetComponent<Rigidbody2D>();

_animator = bear.GetComponentInChildren<Animator>();

_detectionSolDroite =
    bear.GetComponentInChildren<DetectionSolDroite>();

_detectionSolGauche =
    bear.GetComponentInChildren<DetectionSolGauche>();

_canAttack = bear.GetComponent<CanAttack>();
}

/// <summary>
/// Fonction appelée à intervalle fixe.
/// Gère les déplacements physiques de l'ours.
/// </summary>
void FixedUpdate()
{
    float directionX = movex;

    // Bloque le déplacement vers la droite
    // si aucun sol n'est détecté
    if (movex > 0 &&
        !_detectionSolDroite.estAuSolDroite)
    {
        directionX = 0;
    }

    // Bloque le déplacement vers la gauche
    // si aucun sol n'est détecté
    if (movex < 0 &&
        !_detectionSolGauche.estAuSolGauche)
    {
        directionX = 0;
    }

    // Vérifie si l'ours est dans l'eau
    if (IsInWater)
    {
        _rb.gravityScale = 2f;

        // Stop le déplacement
        _rb.linearVelocity = new Vector2(0, 0);

        // Inflige des dégâts dans l'eau
        DamageInTheWater();
    }
    else
    {
        // Gravité normale
        _rb.gravityScale = 1f;
    }
}
```

```
        // Déplacement horizontal
        _rb.linearVelocity =
            new Vector2(
                directionX * _speed,
                _rb.linearVelocity.y
            );
    }
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// </summary>
void Update()
{
    GererAnimation();
}

/// <summary>
/// Fonction appelée automatiquement lors du déplacement.
/// </summary>
/// <param name="value">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnMove(InputValue value)
{
    movex = value.Get<Vector2>().x;
}

/// <summary>
/// Gère les animations de l'ours.
/// </summary>
void GererAnimation()
{
    // Vérifie si l'ours n'est pas blessé
    if (!Hurt)
    {
        // Orientation du sprite vers la droite
        if (movex > 0)
        {
            GetComponentInChildren<SpriteRenderer>().flipX = false;
        }

        // Orientation du sprite vers la gauche
        else if (movex < 0)
        {
            GetComponentInChildren<SpriteRenderer>().flipX = true;
        }

        // Animation de marche
        _animator.SetBool("IsWalking", movex != 0);
    }
    else
    {

```

```
        // Déclenche l'animation de dégâts
        _animator.SetTrigger("Hurt");

        // Réinitialise l'état de blessure
        Hurt = false;
    }
}

/// <summary>
/// Fonction appelée lors de l'attaque.
/// </summary>
/// <param name="value">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnAttack(InputValue value)
{
    // Vérifie si le joueur attaque
    // et qu'aucune attaque n'est déjà en cours
    if (value.isPressed && _isAttacking == false)
    {
        _isAttacking = true;

        // Déclenche l'animation d'attaque
        _animator.SetTrigger("Attack");

        // Réinitialise l'attaque après 0.5 secondes
        Invoke("ResetAttack", 0.5f);
    }
}

/// <summary>
/// Inflige des dégâts à l'ennemi détecté.
/// Fonction appelée via un Animation Event.
/// </summary>
public void EnleverDegat()
{
    _entityEnemy = _canAttack.entityEnemy;

    // Vérifie si un ennemi est détecté
    if (_entityEnemy != null)
    {
        // Inflige des dégâts
        _entityEnemy.TakeDamage(1);

        _entityEnemy = null;
    }
}

/// <summary>
/// Réinitialise l'état d'attaque.
/// </summary>
public void ResetAttack()
{
    _isAttacking = false;
}
```



```
}  
}
```

CanAttack

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Bear/Script/CanAttack.cs

```
//Valentin Tercier  
//07.05.2026  
//Ce script est utilisé pour dire que l'ours peut attaquer si l'ennemie est dans  
la zone d'attack  
using UnityEngine;  
  
public class CanAttack : MonoBehaviour  
{  
    public bool canAttack = false;  
    public Entity entityEnemy;  
    // Start is called once before the first execution of Update after the  
MonoBehaviour is created  
    void Start()  
    {  
  
    }  
  
    // Update is called once per frame  
    void OnTriggerStay2D(Collider2D collision)  
    {  
  
        if (collision.gameObject.tag == "Enemy")  
        {  
            canAttack = true;  
            entityEnemy = collision.gameObject.GetComponentInParent<Entity>();  
        }  
    }  
  
    void OnTriggerExit2D(Collider2D collision)  
    {  
        if (collision.gameObject.tag == "Enemy")  
        {  
            canAttack = false;  
            entityEnemy = null;  
        }  
    }  
}
```

DetectionSolDroite

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Bear/Script/DetectionSolDroite.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour détecter le sol à droite
using UnityEngine;

public class DetectionSolDroite : MonoBehaviour
{
    /// <summary>
    /// Vérifie si un sol est détecté à droite du joueur.
    /// </summary>
    public bool estAuSolDroite;

    /// <summary>
    /// Vérifie si un mur bloque le déplacement.
    /// </summary>
    public bool bloquer;

    /// <summary>
    /// Fonction appelée automatiquement lorsqu'un collider
    /// entre dans la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider détecté par le trigger.
    /// </param>
    void OnTriggerEnter2D(Collider2D collision)
    {
        // Vérifie si l'objet détecté est un mur
        if(collision.gameObject.tag == "Wall")
        {
            bloquer = true;
        }

        // Vérifie si l'objet détecté est le sol
        if (collision.gameObject.tag == "Sol")
        {
            estAuSolDroite = true;
        }
    }

    /// <summary>
    /// Fonction appelée tant qu'un collider reste
    /// dans la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider détecté par le trigger.
    /// </param>
    void OnTriggerStay2D(Collider2D collision)
```

```

    {
        // Vérifie si l'objet détecté est un mur
        if(collision.gameObject.tag == "Wall")
        {
            bloquer = true;
        }

        // Vérifie si l'objet détecté est le sol
        if (collision.gameObject.tag == "Sol")
        {
            estAuSolDroite = true;
        }
    }

    /// <summary>
    /// Fonction appelée lorsqu'un collider
    /// quitte la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider qui quitte le trigger.
    /// </param>
    void OnTriggerExit2D(Collider2D collision)
    {
        // Vérifie si le collider quitté est le sol
        if (collision.gameObject.tag == "Sol")
        {
            estAuSolDroite = false;
        }

        // Vérifie si le collider quitté est un mur
        if(collision.gameObject.tag == "Wall")
        {
            bloquer = false;
        }
    }
}

```

DetectionSolGauche

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Bear/Script/DetectionSolGauche.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour détecter le sol à gauche
using UnityEngine;

public class DetectionSolGauche : MonoBehaviour
{
    public bool estAuSolGauche;
}

```

```
public bool bloquer;  
// Start is called once before the first execution of Update after the  
MonoBehaviour is created  
void OnTriggerEnter2D(Collider2D collision)  
{  
    if(collision.gameObject.tag == "Wall")  
    {  
        bloquer = true;  
    }  
    if (collision.gameObject.tag == "Sol")  
    {  
        estAuSolGauche = true;  
    }  
}  
  
void OnTriggerStay2D(Collider2D collision)  
{  
    if(collision.gameObject.tag == "Wall")  
    {  
        bloquer = true;  
    }  
    if (collision.gameObject.tag == "Sol")  
    {  
        estAuSolGauche = true;  
    }  
}  
  
void OnTriggerExit2D(Collider2D collision)  
{  
    Debug.Log("OnTriggerExit2D");  
    if (collision.gameObject.tag == "Sol")  
    {  
        estAuSolGauche = false;  
    }  
    if(collision.gameObject.tag == "Wall")  
    {  
        bloquer = false;  
    }  
}  
}
```

Bird

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Birds/Script/Bird.cs

```
//Valentin Tercier  
//07.05.2026  
//Ce script est utilisé pour les déplacement de l'oiseau
```

```
using UnityEngine;
using UnityEngine.InputSystem;

public class Bird : Entity
{
    /// <summary>
    /// GameObject principal de l'oiseau.
    /// </summary>
    public GameObject bird;

    /// <summary>
    /// Rigidbody2D utilisé pour gérer les déplacements et la physique.
    /// </summary>
    private Rigidbody2D _rb;

    /// <summary>
    /// Animator utilisé pour gérer les animations.
    /// </summary>
    Animator _anim;

    /// <summary>
    /// Valeur du déplacement vertical récupérée via l'Input System.
    /// </summary>
    float _movey;

    /// <summary>
    /// Vitesse de déplacement de l'oiseau.
    /// </summary>
    public float speed = 3;

    /// <summary>
    /// Vérifie si l'oiseau est en train de voler.
    /// </summary>
    bool _IsFlying;

    /// <summary>
    /// Script permettant de détecter si l'oiseau touche le sol.
    /// </summary>
    DetectionSol _detectionSol;

    /// <summary>
    /// SpriteRenderer utilisé pour retourner le sprite.
    /// </summary>
    SpriteRenderer _sr;

    /// <summary>
    /// Initialisation des composants nécessaires.
    /// </summary>
    void Start()
    {
        _rb = bird.GetComponent<Rigidbody2D>();

        _anim = bird.GetComponentInChildren<Animator>();
    }
}
```

```
        _detectionSol =
            bird.GetComponentInChildren<DetectionSol>();

        _sr = bird.GetComponentInChildren<SpriteRenderer>();

        base.Start();
    }

    /// <summary>
    /// Mise à jour appelée à chaque frame.
    /// Gère les déplacements et les animations.
    /// </summary>
    void Update()
    {
        // Vérifie si l'oiseau est dans l'eau
        if (IsInWater)
        {
            _rb.gravityScale = 1f;

            // Stop le déplacement
            _rb.linearVelocity = new Vector2(0, 0);

            // Inflige des dégâts dans l'eau
            DamageInTheWater();
        }
        else
        {
            // Déplacement horizontal et vertical
            _rb.linearVelocity =
                new Vector2(movex * speed, _movey * speed);
        }

        // Mise à jour des animations
        GererAnimation();
    }

    /// <summary>
    /// Fonction appelée automatiquement lors du déplacement.
    /// </summary>
    /// <param name="inputValue">
    /// Valeur envoyée par le système d'Input.
    /// </param>
    void OnMove(InputValue inputValue)
    {
        Vector2 d = inputValue.Get<Vector2>();

        movex = d.x;

        _movey = d.y;
    }

    /// <summary>
    /// Gère les animations de l'oiseau
    /// selon son état et ses déplacements.
```

```
/// </summary>
void GererAnimation()
{
    // Vérifie si l'oiseau n'est pas blessé
    if (!Hurt)
    {
        // Vérifie si l'oiseau vole
        if (_movey > 0.1 || !_detectionSol.ToucheLeSol)
        {
            _IsFlying = true;
        }

        // Vérifie si l'oiseau touche le sol
        if (_detectionSol.ToucheLeSol)
        {
            _IsFlying = false;
        }

        // Animation de marche
        _anim.SetBool(
            "IsWalking",
            (movex > 0.1 || movex < -0.1) &&
            _IsFlying == false &&
            !IsInWater
        );

        // Animation de vol
        _anim.SetBool(
            "IsFlying",
            _IsFlying && !IsInWater
        );

        // Orientation du sprite vers la droite
        if (movex > 0.1f)
        {
            _sr.flipX = true;
        }

        // Orientation du sprite vers la gauche
        else if (movex < -0.1f)
        {
            _sr.flipX = false;
        }
    }
    else
    {
        // Déclenche l'animation de dégâts
        _anim.SetTrigger("Hurt");

        // Réinitialise l'état de blessure
        Hurt = false;
    }
}
```

DetectionInteraction

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Evil Wizard
3/Script/DetectionInteraction.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour détecter si je peux interagir avec l'objet
using UnityEngine;

public class DetectionInteraction : MonoBehaviour
{
    /// <summary>
    /// Vérifie si un objet interactif est détecté.
    /// </summary>
    public bool IsCollectable;

    /// <summary>
    /// Objet interactif actuellement détecté par le joueur.
    /// </summary>
    public GameObject ObjectToCollect;

    /// <summary>
    /// Initialisation des variables.
    /// </summary>
    void Start()
    {
        IsCollectable = false;
    }

    /// <summary>
    /// Fonction appelée automatiquement lorsqu'un collider
    /// entre dans la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider détecté par le trigger.
    /// </param>
    void OnTriggerEnter2D(Collider2D collision)
    {
        // Vérifie si l'objet possède le tag "Interact"
        if(collision.gameObject.tag == "Interact")
        {
            Debug.Log(
                "Object " +
                collision.gameObject.name +
                " detected"
            );
        }
    }
}
```



```

        // Sauvegarde l'objet détecté
        ObjectToCollect = collision.gameObject;
    }
}

/// <summary>
/// Fonction appelée automatiquement lorsqu'un collider
/// quitte la zone de détection.
/// </summary>
/// <param name="collision">
/// Collider qui quitte le trigger.
/// </param>
void OnTriggerExit2D(Collider2D collision)
{
    // Vérifie si l'objet quitté possède le tag "Interact"
    if(collision.gameObject.tag == "Interact")
    {
        // Réinitialise l'objet interactif détecté
        ObjectToCollect = null;
    }
}
}

```

DetectionSol

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Evil Wizard 3/Script/DetectionSol.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour détecter si il est au sol
using UnityEngine;

public class DetectionSol : MonoBehaviour
{
    /// <summary>
    /// Vérifie si le joueur touche actuellement le sol.
    /// </summary>
    public bool ToucheLeSol;

    /// <summary>
    /// Fonction appelée automatiquement lorsqu'un collider
    /// entre dans la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider détecté par le trigger.
    /// </param>
    private void OnTriggerEnter2D(Collider2D collision)
    {
        // Vérifie si l'objet touché possède le tag "Sol"
    }
}

```

```

        if (collision.CompareTag("Sol"))
        {
            // Le joueur touche le sol
            ToucheLeSol = true;
        }
    }

    /// <summary>
    /// Fonction appelée automatiquement lorsqu'un collider
    /// quitte la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider qui quitte le trigger.
    /// </param>
    private void OnTriggerExit2D(Collider2D collision)
    {
        // Vérifie si l'objet quitté possède le tag "Sol"
        if (collision.CompareTag("Sol"))
        {
            // Le joueur ne touche plus le sol
            ToucheLeSol = false;
        }
    }
}

```

Player

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Evil Wizard 3/Script/Player.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour les mouvement et les interaction du druide
using UnityEngine;
using UnityEngine.InputSystem;

public class Player : Entity
{
    /// <summary>
    /// GameObject principal du druide contrôlé par le joueur.
    /// </summary>
    public GameObject Druide;

    /// <summary>
    /// Rigidbody2D utilisé pour gérer les déplacements et la physique.
    /// </summary>
    Rigidbody2D _rb;

    /// <summary>
    /// Script permettant de détecter si le joueur touche le sol.

```

```
/// </summary>
DetectionSol _detectionSol;

/// <summary>
/// Vitesse de déplacement horizontale du joueur.
/// </summary>
float _speed = 5f;

/// <summary>
/// Force appliquée lors du saut.
/// </summary>
float _jumpForce = 6f;

/// <summary>
/// Vérifie si le joueur a sauté.
/// </summary>
bool _jump = true;

/// <summary>
/// Valeur du déplacement vertical récupérée via l'Input System.
/// </summary>
private float _movey;

/// <summary>
/// Vérifie si le joueur est en train de sauter.
/// </summary>
bool _isJumping;

/// <summary>
/// Vérifie si le joueur est en train de tomber.
/// </summary>
bool _isFalling;

/// <summary>
/// Vérifie si le joueur est en train de marcher.
/// </summary>
bool _isWalking;

/// <summary>
/// Collider principal du joueur.
/// </summary>
CapsuleCollider2D _capsuleCollider2D;

/// <summary>
/// Offset du collider lorsque le joueur regarde à droite.
/// </summary>
float _offsetxRight;

/// <summary>
/// Offset du collider lorsque le joueur regarde à gauche.
/// </summary>
float _offsetxLeft = 0.06f;

/// <summary>
```

```
/// Animator utilisé pour gérer les animations du joueur.
/// </summary>
Animator _anim;

/// <summary>
/// Script permettant de détecter les objets interactifs proches du joueur.
/// </summary>
DetectionInteraction _detectionInteraction;

/// <summary>
/// Préfab du cristal récupérable.
/// </summary>
public GameObject prefabCrystal;

/// <summary>
/// Initialisation des composants nécessaires au joueur.
/// </summary>
void Start()
{
    _rb = Druide.GetComponent<Rigidbody2D>();

    _detectionSol = Druide.GetComponentInChildren<DetectionSol>();

    _capsuleCollider2D = Druide.GetComponent<CapsuleCollider2D>();

    // Sauvegarde de l'offset du collider
    _offsetxRight = _capsuleCollider2D.offset.x;

    _anim = Druide.GetComponentInChildren<Animator>();

    _detectionInteraction =
        Druide.GetComponentInChildren<DetectionInteraction>();

    // Vérifie si le joueur possède déjà le cristal
    if(PlayerStatManager.Instance.HasCrystal)
    {
        GameObject instanceCrystal = Instantiate(prefabCrystal);

        DeplacerCrystal(instanceCrystal);

        instanceCrystal.name = "crystal";
    }

    base.Start();
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Gère les déplacements et les animations.
/// </summary>
void Update()
{
    // Vérifie si le joueur est dans l'eau
```

```
    if (IsInWater)
    {
        _rb.gravityScale = 2f;

        // Stop le déplacement du joueur
        _rb.linearVelocity = new Vector2(0, 0);

        // Inflige des dégâts dans l'eau
        DamageInTheWater();
    }
    else
    {
        // Gravité normale
        _rb.gravityScale = 1f;

        // Déplacement horizontal du joueur
        _rb.linearVelocity =
            new Vector2(movex * _speed, _rb.linearVelocity.y);
    }

    // Mise à jour des animations
    gererAnimation();
}

/// <summary>
/// Fonction appelée automatiquement lors du déplacement du joueur.
/// </summary>
/// <param name="value">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnMove(InputValue value)
{
    Vector2 d = value.Get<Vector2>();

    movex = d.x;
    _movey = d.y;
}

/// <summary>
/// Fonction appelée lorsque le joueur saute.
/// </summary>
/// <param name="value">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnJump(InputValue value)
{
    // Vérifie si le joueur appuie sur la touche
    // et qu'il touche le sol
    if (value.isPressed && _detectionSol.ToucheLeSol)
    {
        // Applique une impulsion vers le haut
        _rb.AddForce(
            new Vector2(0, _jumpForce),
            ForceMode2D.Impulse
        );
    }
}
```

```
        );

        _jump = true;
    }
}

/// <summary>
/// Fonction appelée lors d'une interaction avec un objet.
/// Permet de récupérer un cristal,
/// activer un levier ou déposer le cristal.
/// </summary>
/// <param name="value">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnInteract(InputValue value)
{
    if(value.isPressed)
    {
        // Objet détecté par le joueur
        GameObject objectToCollect =
            _detectionInteraction.ObjectToCollect;

        // Vérifie si l'objet est un cristal
        if(objectToCollect != null &&
            objectToCollect.name == "crystal")
        {
            DeplacerCrystal(objectToCollect);

            PlayerStatManager.Instance.HasCrystal = true;
        }

        // Vérifie si l'objet est un levier
        else if(objectToCollect.name == "Lever" &&
            objectToCollect.CompareTag("Interact"))
        {
            objectToCollect
                .GetComponent<Lever>()
                .isActivated = true;
        }

        // Vérifie si le joueur dépose le cristal
        else if(objectToCollect.name == "depotCrystal" &&
            objectToCollect.CompareTag("Interact"))
        {
            Transform crystal =
                Druide.transform.Find("crystal");

            if(crystal != null)
            {
                PlayerStatManager.Instance.HasCrystal = false;

                // Attache le cristal au dépôt
                crystal.SetParent(objectToCollect.transform);
            }
        }
    }
}
```

```
        else
        {
            Debug.Log("No crystal to deposit");
        }
    }
}

/// <summary>
/// Déplace le cristal sur le joueur
/// et désactive ses composants visuels.
/// </summary>
/// <param name="objectToCollect">
/// Cristal à déplacer.
/// </param>
void DeplacerCristal(GameObject objectToCollect)
{
    // Attache le cristal au joueur
    objectToCollect.transform.SetParent(Druide.transform);

    // Désactive le rendu du cristal
    objectToCollect
        .GetComponentInChildren<SpriteRenderer>()
        .enabled = false;

    // Désactive son collider
    objectToCollect
        .GetComponent<Collider2D>()
        .enabled = false;

    // Désactive la lumière du cristal
    objectToCollect
        .transform
        .Find("Spot Light 2D")
        .gameObject
        .SetActive(false);
}

/// <summary>
/// Gère les animations du joueur
/// selon ses déplacements et son état.
/// </summary>
public void gererAnimation()
{
    // Vérifie si le joueur n'est pas blessé
    if (!Hurt)
    {
        // Direction vers la droite
        if (movex > 0)
        {
            GetComponentInChildren<SpriteRenderer>().flipX = false;

            _capsuleCollider2D.offset =
                new Vector2(
```

```
        _offsetxRight,
        _capsuleCollider2D.offset.y
    );
}

// Direction vers la gauche
else if (movex < 0)
{
    GetComponentInChildren<SpriteRenderer>().flipX = true;

    _capsuleCollider2D.offset =
        new Vector2(
            _offsetxLeft,
            _capsuleCollider2D.offset.y
        );
}

// Détection des états du joueur
_isJumping = _rb.linearVelocity.y > 0.1f;

_isFalling = _rb.linearVelocity.y < -0.1f;

_isWalking = movex != 0;

// Mise à jour des paramètres Animator
_anim.SetBool("IsJumping", _isJumping);

_anim.SetBool("IsFalling", _isFalling);

_anim.SetBool(
    "IsWalking",
    _isWalking && !_isJumping && !_isFalling
);
}
else
{
    // Déclenche l'animation de dégâts
    _anim.SetTrigger("Hurt");

    Hurt = false;
}
}
}
```

Fish

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/fish/Script/Fish.cs


```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour les mouvement du poisson
using System;
using UnityEngine;
using UnityEngine.InputSystem;

public class Fish : Entity
{
    /// <summary>
    /// GameObject principal du poisson.
    /// </summary>
    public GameObject fish;

    /// <summary>
    /// Rigidbody2D utilisé pour gérer les déplacements et la physique.
    /// </summary>
    Rigidbody2D _rb;

    /// <summary>
    /// Animator utilisé pour gérer les animations.
    /// </summary>
    Animator _anim;

    /// <summary>
    /// Valeur du déplacement vertical récupérée via l'Input System.
    /// </summary>
    float _movey;

    /// <summary>
    /// Vitesse maximale du poisson.
    /// </summary>
    float _speed = 7f;

    /// <summary>
    /// Vitesse d'accélération du poisson.
    /// </summary>
    float _acceleration = 4f;

    /// <summary>
    /// Vitesse actuelle utilisée pour lisser le déplacement.
    /// </summary>
    Vector2 _velocityCourante;

    /// <summary>
    /// Vitesse de rotation du poisson.
    /// </summary>
    float _rotationSpeed = 200f;

    /// <summary>
    /// SpriteRenderer utilisé pour retourner le sprite.
    /// </summary>
    SpriteRenderer _spriteRenderer;
```

```
/// <summary>
/// Initialisation des composants nécessaires.
/// </summary>
void Start()
{
    _rb = fish.GetComponent<Rigidbody2D>();

    _anim = fish.GetComponentInChildren<Animator>();

    _spriteRenderer =
        fish.GetComponentInChildren<SpriteRenderer>();

    base.Start();
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Gère les déplacements et les animations.
/// </summary>
void Update()
{
    // Vérifie si le poisson est dans l'eau
    if(IsInWater)
    {
        // Désactive la gravité
        _rb.gravityScale = 0f;

        // Calcul de la vitesse souhaitée
        Vector2 VitesseVoulue =
            new Vector2(
                movex * _speed,
                _movey * _speed
            );

        // Lissage du déplacement
        _velocityCourante =
            Vector2.Lerp(
                _velocityCourante,
                VitesseVoulue,
                _acceleration * Time.deltaTime
            );

        _rb.linearVelocity = _velocityCourante;

        // Vérifie si le poisson est en mouvement
        if(Mathf.Abs(movex) > 0.1 ||
            Mathf.Abs(_movey) > 0.1)
        {
            // Calcul de l'angle visé
            float angleVise =
                Mathf.Atan2(_movey, movex) *
                Mathf.Rad2Deg;
        }
    }
}
```

```

        // Angle actuel du poisson
        float angleActuel =
            fish.transform.rotation.eulerAngles.z;

        // Rotation progressive
        float nouvelAngle =
            Mathf.MoveTowardsAngle(
                angleActuel,
                angleVise,
                _rotationSpeed * Time.deltaTime
            );

        // Application de la rotation
        fish.transform.rotation =
            Quaternion.Euler(0, 0, nouvelAngle);
    }
}
else
{
    // Réactive la gravité hors de l'eau
    _rb.gravityScale = 1f;

    // Stop le déplacement horizontal
    _rb.linearVelocity =
        new Vector2(0, _rb.linearVelocityY);
}

// Mise à jour des animations
GererAnimation();
}

/// <summary>
/// Fonction appelée automatiquement lors du déplacement.
/// </summary>
/// <param name="inputValue">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnMove(InputValue inputValue)
{
    Vector2 d = inputValue.Get<Vector2>();

    movex = d.x;

    _movey = d.y;
}

/// <summary>
/// Gère les animations et l'orientation du sprite.
/// </summary>
void GererAnimation()
{
    // Vérifie si le poisson est orienté vers le bas
    if(fish.transform.rotation.eulerAngles.z > 90 &&
        fish.transform.rotation.eulerAngles.z < 270)

```

```

        {
            _spriteRenderer.flipY = true;
        }
        else
        {
            _spriteRenderer.flipY = false;
        }

        // Animation de nage
        //anim.SetBool(
        //    "IsSwimming",
        //    (movex > 0.1 ||
        //     movex < -0.1 ||
        //     _movey > 0.1 ||
        //     _movey < -0.1) &&
        //    IsInWater
        //);
    }
}

```

Entity

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Script/Entity.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour toute les classe descendante de entity
using System.Threading;
using NUnit.Framework;
using UnityEngine;

public class Entity : MonoBehaviour
{
    /// <summary>
    /// Référence au script GestionPersonnage.
    /// Permet notamment de récupérer la vie maximale
    /// et de mettre à jour l'interface.
    /// </summary>
    private GestionPersonnage _gp;

    /// <summary>
    /// Vie actuelle de l'entité.
    /// </summary>
    public int currentHealth;

    /// <summary>
    /// Valeur du déplacement horizontal.
    /// </summary>
    public float movex;
}

```

```
/// <summary>
/// Vérifie si l'entité vient de recevoir des dégâts.
/// </summary>
public bool Hurt;

/// <summary>
/// Indique si l'entité possède une animation de mort.
/// </summary>
public bool HaveDeadAnimation = false;

/// <summary>
/// Vérifie si l'entité est morte.
/// </summary>
public bool IsDead;

/// <summary>
/// Vérifie si l'entité est actuellement dans l'eau.
/// </summary>
public bool IsInWater = false;

/// <summary>
/// Timer utilisé pour appliquer des dégâts dans l'eau.
/// </summary>
float _timerInWater = 0f;

/// <summary>
/// Temps maximum avant d'infliger des dégâts dans l'eau.
/// </summary>
float _timerInWaterMax = 0.5f;

/// <summary>
/// Initialisation de l'entité.
/// </summary>
public void Start()
{
    IsDead = false;

    _gp = GetComponent<GestionPersonnage>();

    currentHealth = _gp.MaxHealth;
}

/// <summary>
/// Inflige des dégâts à l'entité.
/// </summary>
/// <param name="damage">
/// Nombre de points de vie à retirer.
/// </param>
public void TakeDamage(int damage)
{
    currentHealth -= damage;

    Hurt = true;
```

```
// Met à jour l'interface de vie et de mana
if(_gp != null)
{
    _gp.MettreAJourVieEtMana(currentHealth);
}

// Vérifie si l'entité n'a plus de vie
if (currentHealth <= 0)
{
    Dead();
}
}

/// <summary>
/// Gère la mort de l'entité.
/// </summary>
private void Dead()
{
    // Si aucune animation de mort n'est prévue,
    // l'objet est détruit après un court délai
    if (!HaveDeadAnimation)
    {
        Invoke("LancerAnimation", 0.5f);
    }

    IsDead = true;
}

/// <summary>
/// Détruit le GameObject de l'entité.
/// </summary>
private void LancerAnimation()
{
    Destroy(gameObject);
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// </summary>
void Update()
{
}

/// <summary>
/// Inflige régulièrement des dégâts
/// lorsque l'entité reste dans l'eau.
/// </summary>
public void DamageInTheWater()
{
    _timerInWater += Time.deltaTime;

    // Vérifie si le temps avant dégâts est atteint
```

```
        if(_timerInWater >= _timerInWaterMax)
        {
            TakeDamage(5);

            _timerInWater = 0;
        }
    }
}
```

GestionPersonnage

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Character/Script/GestionPersonnage.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé gérer la transformation entre les personnage et le mana
entre les personnages
using TMPro;
using UnityEngine;
using UnityEngine.InputSystem;

public class GestionPersonnage : MonoBehaviour
{
    /// <summary>
    /// Liste des différentes formes disponibles pour le personnage.
    /// </summary>
    public enum TypePersonnage
    {
        Druide,
        Bears,
        Fish,
        Bird
    }

    /// <summary>
    /// Vie maximale du personnage.
    /// </summary>
    public int MaxHealth = 100;

    /// <summary>
    /// Vie actuelle du personnage.
    /// </summary>
    private int _currentHealth;

    /// <summary>
    /// Mana maximale du personnage.
    /// </summary>
    public int MaxMana = 100;
```

```
/// <summary>
/// Mana actuelle du personnage.
/// </summary>
private int _currentMana;

/// <summary>
/// Forme actuellement utilisée par le joueur.
/// </summary>
public TypePersonnage currentPersonnage;

/// <summary>
/// GameObject de la forme ours.
/// </summary>
public GameObject BearGo;

/// <summary>
/// GameObject de la forme druide.
/// </summary>
public GameObject DruideGo;

/// <summary>
/// GameObject de la forme poisson.
/// </summary>
public GameObject FishGo;

/// <summary>
/// GameObject de la forme oiseau.
/// </summary>
public GameObject BirdGo;

/// <summary>
/// Position du VFX pour la transformation en ours.
/// </summary>
public GameObject VfxPositionBear;

/// <summary>
/// Position du VFX pour la transformation en druide.
/// </summary>
public GameObject VfxPositionDruide;

/// <summary>
/// Position du VFX pour la transformation en poisson.
/// </summary>
public GameObject VfxPositionFish;

/// <summary>
/// Position du VFX pour la transformation en oiseau.
/// </summary>
public GameObject VfxPositionBird;

/// <summary>
/// Effet visuel joué lors d'une transformation.
/// </summary>
public GameObject Vfx;
```



```
/// <summary>
/// Référence au script de l'ours.
/// </summary>
private Bear _bear;

/// <summary>
/// Référence au script du druide.
/// </summary>
private Player _Druide;

/// <summary>
/// Référence au script de l'oiseau.
/// </summary>
private Bird _bird;

/// <summary>
/// Référence au script du poisson.
/// </summary>
private Fish _fish;

/// <summary>
/// Position actuelle du personnage avant une transformation.
/// </summary>
Vector2 _positionActuelle;

/// <summary>
/// Animator du VFX de transformation.
/// </summary>
Animator _animVfx;

//timer

/// <summary>
/// Timer utilisé pour gérer la consommation ou la récupération de mana.
/// </summary>
float _timer = 0f;

/// <summary>
/// Temps maximum avant de modifier la mana.
/// </summary>
float _timerMax = 1f;

/// <summary>
/// Texte affichant la vie dans l'interface.
/// </summary>
public GameObject vie;

/// <summary>
/// Texte affichant la mana dans l'interface.
/// </summary>
public GameObject mana;

private bool _isDead;
```

```
/// <summary>
/// Initialisation des formes, des statistiques et du VFX.
/// </summary>
void Start()
{
    currentPersonnage = TypePersonnage.Druide;
    _bear = GetComponent<Bear>();
    _Druide = GetComponent<Player>();
    _bird = GetComponent<Bird>();
    _fish = GetComponent<Fish>();
    _animVfx = Vfx.GetComponentInChildren<Animator>();
    _currentHealth = PlayerStatManager.Instance.health;
    _currentMana = PlayerStatManager.Instance.mana;
    MaxHealth = PlayerStatManager.Instance.maxHealth;
    MaxMana = PlayerStatManager.Instance.maxMana;
    if(_currentHealth == 0)
    {
        _currentHealth = MaxHealth;
    }
    Vfx.SetActive(false);
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Gère les transformations, la mana, l'interface et la sauvegarde des
statistiques.
/// </summary>
void Update()
{
    if(_currentHealth <= 0)
    {
        _isDead = true;
    }
    // Si la mana est vide, le joueur retourne automatiquement en druide
    if(_currentMana <= 0)
    {
        currentPersonnage = TypePersonnage.Druide;
        _currentMana += 1;
        _DruideGo.transform.position = new Vector2(_positionActuelle.x,
_positionActuelle.y);

        Vfx.SetActive(true);
        Vfx.transform.position = new
Vector2(VfxPositionDruide.transform.position.x,
VfxPositionDruide.transform.position.y);
    }

    switch (currentPersonnage)
    {
```

```
case TypePersonnage.Druide:
    //récupérer position actuelle
    _positionActuelle = DruideGo.transform.position;
    //StatDeVie

    _Druide.currentHealth = _currentHealth;

    //script
    _Druide.enabled = true;
    _bear.enabled = false;
    _bird.enabled = false;
    _fish.enabled = false;
    //gameObject
    DruideGo.SetActive(true);
    BearGo.SetActive(false);
    FishGo.SetActive(false);
    BirdGo.SetActive(false);

    break;
case TypePersonnage.Bears:
    //récupérer position actuelle
    _positionActuelle = BearGo.transform.position;
    //StatDeVie

    _bear.currentHealth = _currentHealth;

    //script
    _Druide.enabled = false;
    _bear.enabled = true;
    _bird.enabled = false;
    _fish.enabled = false;
    //gameObject

    BearGo.SetActive(true);
    DruideGo.SetActive(false);
    FishGo.SetActive(false);
    BirdGo.SetActive(false);

    // Handle Bears specific logic
    break;
case TypePersonnage.Fish:
    //récupérer position actuelle
    _positionActuelle = FishGo.transform.position;
    //Stat de vie et de mana
    _fish.currentHealth = _currentHealth;
    // script
    _Druide.enabled = false;
    _bear.enabled = false;
    _bird.enabled = false;
    _fish.enabled = true;
    // GameObject
    FishGo.SetActive(true);
    BearGo.SetActive(false);
```

```
        BirdGo.SetActive(false);
        DruideGo.SetActive(false);
        // Handle Fish specific logic
        break;
    case TypePersonnage.Bird:
        //récupérer position actuelle
        _positionActuelle = BirdGo.transform.position;
        //Stat de vie et de mana

        _bird.currentHealth = _currentHealth;

        //script
        _bird.enabled = true;
        _Druide.enabled = false;
        _bear.enabled = false;
        _fish.enabled = false;
        //GameObject
        BirdGo.SetActive(true);
        FishGo.SetActive(false);
        BearGo.SetActive(false);
        DruideGo.SetActive(false);
        // Handle Bird specific logic
        break;
}

_timer += Time.deltaTime;
if (_timer >= _timerMax)
{
    //Debug.Log("une seconde");

    // Consommation de mana en forme ours
    if (currentPersonnage == TypePersonnage.Bears)
    {
        _currentMana -= 6;
    }

    // Consommation de mana en forme poisson
    else if (currentPersonnage == TypePersonnage.Fish)
    {
        _currentMana -= 8;
    }

    // Consommation de mana en forme oiseau
    else if (currentPersonnage == TypePersonnage.Bird)
    {
        _currentMana -= 8;
    }

    // Régénération de mana en forme druide
    else if (currentPersonnage == TypePersonnage.Druide && _currentMana <
MaxMana)
    {
        _currentMana += 10;
        if(_currentMana >= MaxMana)
```

```
        {
            _currentMana = MaxMana;
        }
    }

    _timer = 0;
}

afficherVieEtMana();

// Sauvegarde les statistiques actuelles

PlayerStatManager.Instance.SaveStats(_currentHealth,_currentMana,MaxHealth,MaxMana
);
}

// sers à mettre a jour depuis la classe entity
/// <summary>
/// Met à jour la vie actuelle depuis la classe Entity.
/// </summary>
/// <param name="health">
/// Nouvelle valeur de vie actuelle.
/// </param>
public void MettreAJourVieEtMana(int health)
{
    _currentHealth = health;
}

/// <summary>
/// Récupère de la mana.
/// Si la mana dépasse le maximum, le maximum est augmenté.
/// </summary>
public void ManaRecuperer()
{
    MaxMana += 20;
    if (_currentMana > MaxMana)
    {
        int difference = _currentMana - MaxMana;
        MaxMana += difference;
    }
}

/// <summary>
/// Récupère de la vie.
/// Si la vie dépasse le maximum, le maximum est augmenté.
/// </summary>
public void VieRecuperer()
{
    _currentHealth += 20;
    if (_currentHealth > MaxHealth)
    {
        int difference = _currentHealth - MaxHealth;
        MaxHealth += difference;
    }
}
```

```
}

/// <summary>
/// Transforme le personnage en druide.
/// </summary>
/// <param name="inputValue">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnChangeDruide(InputValue inputValue)
{
    if (inputValue.isPressed && TypePersonnage.Druide != currentPersonnage)
    {
        currentPersonnage = TypePersonnage.Druide;
        DruideGo.transform.position = new Vector2(_positionActuelle.x,
        _positionActuelle.y);

        Vfx.SetActive(true);
        Vfx.transform.position = new
        Vector2(VfxPositionDruide.transform.position.x,
        VfxPositionDruide.transform.position.y);
    }
}

/// <summary>
/// Transforme le personnage en ours si la mana est suffisante.
/// </summary>
/// <param name="inputValue">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnChangeBear(InputValue inputValue)
{
    if (inputValue.isPressed && TypePersonnage.Bears != currentPersonnage &&
    _currentMana >= 6)
    {
        currentPersonnage = TypePersonnage.Bears;
        BearGo.transform.position = new Vector2(_positionActuelle.x,
        _positionActuelle.y);

        Vfx.SetActive(true);
        Vfx.transform.position = VfxPositionBear.transform.position;
    }
}

/// <summary>
/// Transforme le personnage en poisson si la mana est suffisante.
/// </summary>
/// <param name="inputValue">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnChangeFish(InputValue inputValue)
{
    if (inputValue.isPressed && TypePersonnage.Fish != currentPersonnage &&
    _currentMana >= 5)
```

```

        {
            currentPersonnage = TypePersonnage.Fish;
            FishGo.transform.position = new Vector2(_positionActuelle.x,
            _positionActuelle.y);

            Vfx.SetActive(true);
            Vfx.transform.position = VfxPositionFish.transform.position;
        }
    }

    /// <summary>
    /// Transforme le personnage en oiseau si la mana est suffisante.
    /// </summary>
    /// <param name="inputValue">
    /// Valeur envoyée par le système d'Input.
    /// </param>
    void OnChangeBird(InputValue inputValue)
    {
        if (inputValue.isPressed && TypePersonnage.Bird != currentPersonnage &&
        _currentMana >= 8)
        {
            currentPersonnage = TypePersonnage.Bird;
            BirdGo.transform.position = new Vector2(_positionActuelle.x,
            _positionActuelle.y);

            Vfx.SetActive(true);
            Vfx.transform.position = VfxPositionBird.transform.position;
        }
    }

    /// <summary>
    /// Désactive le VFX lorsque son animation est terminée.
    /// </summary>
    public void VfxTerminer()
    {
        Vfx.SetActive(false);
    }

    //il sert à afficher la vie pout l'ui
    /// <summary>
    /// Affiche la vie et la mana dans l'interface utilisateur.
    /// </summary>
    void afficherVieEtMana()
    {
        vie.GetComponent<TextMeshProUGUI>().text = _currentHealth + " / " +
MaxHealth;
        mana.GetComponent<TextMeshProUGUI>().text = _currentMana + " / " +
MaxMana;
    }

    /// <summary>
    /// Retourne la vie actuelle du personnage.
    /// </summary>
    /// <returns>

```

```
    /// Vie actuelle.  
    /// </returns>  
    public int RetournerVie()  
    {  
        return _currentHealth;  
    }  
  
    /// <summary>  
    /// Retourne la mana actuelle du personnage.  
    /// </summary>  
    /// <returns>  
    /// Mana actuelle.  
    /// </returns>  
    public int RetournerMana()  
    {  
        return _currentMana;  
    }  
  
    public bool IsDead()  
    {  
        return _isDead;  
    }  
  
}
```

Water

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Environment/Script/Water.cs

```
//Valentin Tercier  
//07.05.2026  
//Ce script est utilisé pour gérer les transformation quand il rentre dans l'eau  
using UnityEngine;  
using UnityEngine.InputSystem;  
  
public class Water : MonoBehaviour  
{  
    /// <summary>  
    /// GameObject actuellement détecté dans l'eau.  
    /// </summary>  
    GameObject _Entity;  
  
    /// <summary>  
    /// Référence au script GestionPersonnage.  
    /// </summary>  
    GestionPersonnage _gp;  
  
    /// <summary>
```



```
/// GameObject du druide.  
/// </summary>  
public GameObject DruideGo;  
  
/// <summary>  
/// GameObject de l'ours.  
/// </summary>  
public GameObject OursGo;  
  
/// <summary>  
/// GameObject du poisson.  
/// </summary>  
public GameObject FishGo;  
  
/// <summary>  
/// GameObject de l'oiseau.  
/// </summary>  
public GameObject BirdGo;  
  
/// <summary>  
/// Référence au script Player du druide.  
/// </summary>  
Player _DruideCs;  
  
/// <summary>  
/// Référence au script Bird.  
/// </summary>  
Bird _birdCs;  
  
/// <summary>  
/// Référence au script Bear.  
/// </summary>  
Bear _bearCs;  
  
/// <summary>  
/// Référence au script Fish.  
/// </summary>  
Fish _fishCs;  
  
/// <summary>  
/// Entité actuellement dans l'eau.  
/// </summary>  
Entity _entite;  
  
/// <summary>  
/// Rigidbody2D de l'entité détectée.  
/// </summary>  
Rigidbody2D _rg;  
  
/// <summary>  
/// Valeur du déplacement horizontal.  
/// </summary>  
float _movex;
```

```
/// <summary>
/// Valeur du déplacement vertical.
/// </summary>
float _movey;

// Start is called once before the first execution of Update after the
MonoBehaviour is created
void Start()
{

}

// Update is called once per frame
void Update()
{

}

/// <summary>
/// Récupère les valeurs de déplacement envoyées par l'Input System.
/// </summary>
/// <param name="inputValue">
/// Valeur envoyée par le système d'Input.
/// </param>
void OnMove(InputValue inputValue)
{
    Vector2 d = inputValue.Get<Vector2>();
    _movex = d.x;
    _movey = d.y;
}

/// <summary>
/// Fonction appelée lorsqu'un objet entre dans l'eau.
/// </summary>
/// <param name="collision">
/// Collider de l'objet qui entre dans le trigger.
/// </param>
void OnTriggerEnter2D(Collider2D collision)
{
    Debug.Log("OnTriggerStay2D" + collision.gameObject.name);
    _Entity = collision.gameObject;

    // Vérifie si l'objet détecté est l'ours
    if(_Entity.name == "Ours")
    {
        _bearCs = _Entity.GetComponentInParent<Bear>();
        _entite = _bearCs;
        _entite.IsInWater = true;
    }

    // Vérifie si l'objet détecté est le druide
    if(_Entity.name == "Druide")
    {
        _DruideCs = _Entity.GetComponentInParent<Player>();
    }
}
```

```
        _entite = _DruideCs;
        _entite.IsInWater = true;
    }

    // Vérifie si l'objet détecté est le poisson
    if(_Entity.name == "Poisson")
    {
        _fishCs = _Entity.GetComponentInParent<Fish>();
        _entite = _fishCs;
        _entite.IsInWater = true;
    }

    // Vérifie si l'objet détecté est l'oiseau
    if(_Entity.name == "Oiseau")
    {
        _birdCs = _Entity.GetComponentInParent<Bird>();
        _entite = _birdCs;
        _entite.IsInWater = true;
    }

    // Vérifie si l'objet détecté est l'abeille
    if(_Entity.name == "Abeille")
    {
        _entite = _Entity.GetComponent<Entity>();
        _entite.IsInWater = true;
    }
}

/// <summary>
/// Fonction appelée tant qu'un objet reste dans l'eau.
/// </summary>
/// <param name="collision">
/// Collider de l'objet présent dans le trigger.
/// </param>
void OnTriggerStay2D(Collider2D collision)
{
    _Entity = collision.gameObject;

    // Vérifie si l'objet détecté est l'ours
    if(_Entity.name == "Ours")
    {
        _bearCs = _Entity.GetComponentInParent<Bear>();
        _entite = _bearCs;
        _entite.IsInWater = true;
    }

    // Vérifie si l'objet détecté est le druide
    if(_Entity.name == "Druide")
    {
        _DruideCs = _Entity.GetComponentInParent<Player>();
        _entite = _DruideCs;
        _entite.IsInWater = true;
    }
}
```

```

        // Vérifie si l'objet détecté est le poisson
        if(_Entity.name == "Poisson")
        {
            _fishCs = _Entity.GetComponentInParent<Fish>();
            _entite = _fishCs;
            _entite.IsInWater = true;
        }

        // Vérifie si l'objet détecté est l'oiseau
        if(_Entity.name == "Oiseau")
        {
            _birdCs = _Entity.GetComponentInParent<Bird>();
            _entite = _birdCs;
            _entite.IsInWater = true;
        }
    }

    /// <summary>
    /// Fonction appelée lorsqu'un objet quitte l'eau.
    /// </summary>
    /// <param name="collision">
    /// Collider de l'objet qui quitte le trigger.
    /// </param>
    void OnTriggerExit2D(Collider2D collision)
    {
        // Vérifie si une entité était détectée
        if (_entite != null)
        {
            _entite.IsInWater = false;
            _Entity = null;
            _entite = null;
        }
    }
}

```

DetectorPlayer

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Legacy-Fantasy/Mob/Boar/Script/DetectorPlayer.cs

```

using UnityEngine;

public class DetectorPlayer : MonoBehaviour
{
    //Valentin Tercier
    //07.05.2026
    //Ce script est utilisé pour détecter si le joueur est dans la zone
    float direction = 1f;
    public Transform playerPosition;
}

```

```
public bool chasePlayer = false;

Sanglier sangliers;

// Start is called once before the first execution of Update after the
MonoBehaviour is created
void Start()
{
    sangliers = GetComponentInParent<Sanglier>();
    direction = sangliers.direction;
}

// Update is called once per frame
void Update()
{
    direction = sangliers.direction;
}

void OnTriggerEnter2D(Collider2D collision)
{
    if (collision.gameObject.tag == "Player")
    {
        if (PlayerIsInFront(collision.transform))
        {
            playerPosition = collision.transform;
            chasePlayer = true;
        }
    }
}

void OnTriggerStay2D(Collider2D collision)
{
    if (collision.gameObject.tag == "Player")
    {
        if (PlayerIsInFront(collision.transform))
        {
            chasePlayer = true;
            playerPosition = collision.transform;
        }
    }
}

bool PlayerIsInFront(Transform playerTransform)
{
    if (direction == 1f && playerTransform.position.x > transform.position.x)
    {
        return true;
    }
}
```

```
        else if (direction == -1f && playerTransform.position.x <
transform.position.x)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
}
```

Sanglier

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Legacy-Fantasy/Mob/Boar/Script/Sanglier.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour le déplacement, les attaques du sanglier
using UnityEngine;
using UnityEngine;

public class Sanglier : Entity
{
    /// <summary>
    /// États possibles du sanglier.
    /// </summary>
    public enum State
    {
        Patrouille,
        Chase,
        Stunt,
    }

    //variable mouvement

    /// <summary>
    /// Vitesse actuelle du sanglier.
    /// </summary>
    public float speed;

    /// <summary>
    /// Vitesse utilisée lorsque le sanglier poursuit le joueur.
    /// </summary>
    public float chaseSpeed = 4f;
```

```
/// <summary>
/// Vitesse utilisée lorsque le sanglier patrouille.
/// </summary>
public float walkSpeed = 2f;

/// <summary>
/// État actuel du sanglier.
/// </summary>
public State currentState;

/// <summary>
/// Direction actuelle du déplacement.
/// </summary>
public float direction;

/// <summary>
/// Rigidbody2D utilisé pour gérer les déplacements et la physique.
/// </summary>
Rigidbody2D _rb;

/// <summary>
/// Animator utilisé pour gérer les animations.
/// </summary>
Animator _anim;

//Script Detection

/// <summary>
/// Détecteur du sol et des murs à droite.
/// </summary>
private DetectionSolDroite _detectionDroite;

/// <summary>
/// Détecteur du sol et des murs à gauche.
/// </summary>
private DetectionSolGauche _detectionGauche;

//SpriteRenderer spriteRenderer;

/// <summary>
/// SpriteRenderer utilisé pour retourner le sprite.
/// </summary>
SpriteRenderer _sr;

/// <summary>
/// GameObject du joueur touché ou détecté.
/// </summary>
GameObject _player;

/// <summary>
/// Entité du joueur permettant d'infliger des dégâts.
/// </summary>
Entity _entity;
```

```
//compteur chase

/// <summary>
/// Timer de poursuite du joueur.
/// </summary>
float _chaseTime = 0f;

/// <summary>
/// Temps maximum pendant lequel le sanglier poursuit le joueur.
/// </summary>
float _maxChaseTime = 3f;

//compteur stunt

/// <summary>
/// Timer de l'état étourdi.
/// </summary>
float _stuntTime = 0f;

/// <summary>
/// Temps maximum pendant lequel le sanglier reste étourdi.
/// </summary>
float _maxStuntTime = 3f;

/// <summary>
/// Script permettant de détecter le joueur.
/// </summary>
DetectorPlayer _detecterPlayer;

/// <summary>
/// Initialisation des composants nécessaires.
/// </summary>
void Start()
{
    IsDead = false;
    _rb = GetComponent<Rigidbody2D>();
    currentState = State.Patrouille;
    direction = 1f;
    _detectionDroite = GetComponentInChildren<DetectionSolDroite>();
    _detectionGauche = GetComponentInChildren<DetectionSolGauche>();
    _anim = GetComponentInChildren<Animator>();
    _sr = GetComponentInChildren<SpriteRenderer>();
    _detecterPlayer = GetComponentInChildren<DetectorPlayer>();
    speed = walkSpeed;
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Gère les états, les déplacements et les animations du sanglier.
/// </summary>
void Update()
{
    // Si le joueur est détecté et que le sanglier n'est pas étourdi,
    // il passe en mode poursuite.
```



```
        if (_detectorPlayer.chasePlayer && currentState != State.Stunt)
        {
            currentState = State.Chase;
            speed = chaseSpeed;
        }

        //Debug.Log("Current State: " + currentState);

        switch (currentState)
        {
            case State.Patrouille:
                Patrouille();
                break;
            case State.Chase:
                Chase();
                break;
            case State.Stunt:
                Stunt();
                break;
        }

        GererAnimation();
    }

    /// <summary>
    /// Gère le déplacement de patrouille du sanglier.
    /// </summary>
    void Patrouille()
    {
        verifierSiIlestAuSolOuEstBloqueParUnMur();
        Flip();
        _rb.linearVelocity = new Vector2(direction * speed, _rb.linearVelocity.y);
    }

    /// <summary>
    /// Vérifie si le sanglier est encore au sol
    /// ou s'il est bloqué par un mur.
    /// </summary>
    void verifierSiIlestAuSolOuEstBloqueParUnMur()
    {
        if (_detectionDroite.estAuSolDroite == true &&
            _detectionGauche.estAuSolGauche == false)
        {
            direction = 1f;
        }
        else if (_detectionDroite.estAuSolDroite == false &&
            _detectionGauche.estAuSolGauche == true)
        {
            direction = -1f;
        }
        if(_detectionDroite.bloquer == true)
        {

```

```
        direction = -1f;
    }
    else if(_detectionGauche.bloquer == true)
    {
        direction = 1f;
    }
}

/// <summary>
/// Retourne le sprite selon la direction du sanglier.
/// </summary>
void Flip()
{
    if (direction == 1f)
    {
        _sr.flipX = true;
    }
    else
    {
        _sr.flipX = false;
    }
}

/// <summary>
/// Gère la poursuite du joueur.
/// </summary>
void Chase()
{
    _rb.linearVelocity = new Vector2(direction * speed, _rb.linearVelocity.y);
    Flip();

    if (_chaseTime < _maxChaseTime)
    {
        _chaseTime += Time.deltaTime;
    }
    else
    {
        _detectorPlayer.chasePlayer = false;
        currentState = State.Patrouille;
        speed = walkSpeed;
        _chaseTime = 0f;
    }
}

/// <summary>
/// Gère l'état étourdi du sanglier.
/// </summary>
void Stunt()
{
    _rb.linearVelocity = Vector2.zero;
    _detectorPlayer.chasePlayer = false;
    if (_stuntTime < _maxStuntTime)
    {

```

```
        _stuntTime += Time.deltaTime;
    }
    else
    {
        currentState = State.Patrouille;
        _stuntTime = 0f;
    }
}

/// <summary>
/// Gère les animations du sanglier selon son état.
/// </summary>
void GererAnimation()
{
    if (!Hurt)
    {
        if (currentState == State.Patrouille)
        {
            _anim.SetBool("isWalking", true);
            _anim.SetBool("isRunning", false);
        }
        else if (currentState == State.Chase)
        {
            _anim.SetBool("isRunning", true);
            _anim.SetBool("isWalking", false);
        }
        else if (currentState == State.Stunt)
        {
            _anim.SetBool("isWalking", false);
            _anim.SetBool("isRunning", false);
        }
    }
    else
    {
        _anim.SetTrigger("Hurt");
        Hurt = false;
    }
}

//fonction pour savoir si il rentre en collision avec le joueur ou un mur
//si cette un joueur il lui inflige des dégats et le repousse sinon il rentre
en stunt

/// <summary>
/// Fonction appelée lorsqu'une collision commence.
/// Permet d'infliger des dégâts au joueur ou
/// de mettre le sanglier en état étourdi.
/// </summary>
/// <param name="collision">
/// Collision détectée.
/// </param>
void OnCollisionEnter2D(Collision2D collision)
{
    Debug.Log("Collision avec: " + collision.gameObject.name + " tag: " +
```

```
collision.gameObject.tag);
    if (collision.gameObject.tag == "Player" && currentState == State.Chase)
    {
        _player = collision.gameObject;
        if (_player.name == "Ours")
        {
            _entity = _player.GetComponentInParent<Bear>();
        }
        if (_player.name == "Druide")
        {
            _entity = _player.GetComponentInParent<Player>();
        }
        if (_player.name == "Poisson")
        {
            _entity = _player.GetComponentInParent<Fish>();
        }
        if (_player.name == "Oiseau")
        {
            _entity = _player.GetComponentInParent<Bird>();
        }

        if (_entity != null && _entity.Hurt == false)
        {
            //Debug.Log("Player touché par le sanglier");
            EnleverDegat();
        }
    }
    if (collision.gameObject.tag == "Wall" && currentState == State.Chase)
    {
        Debug.Log("tape contre le mur");
        currentState = State.Stunt;
    }
}

//fonction pour enlever les dégats au player

/// <summary>
/// Inflige des dégâts au joueur détecté
/// et le repousse avec une force.
/// </summary>
public void EnleverDegat()
{
    if (_entity != null)
    {
        _entity.TakeDamage(1);
        _player.GetComponent<Rigidbody2D>().AddForce(new Vector2(direction *
200, 10), ForceMode2D.Impulse);
        _entity = null;
    }
}
```

```
        else
        {
            Debug.Log("Player est null");
        }
    }
}
```

Abeille

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Legacy-Fantasy/Mob/Small Bee/Script/Abeille.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour L'abeille
using UnityEngine;

public class Abeille : Entity
{
    /// <summary>
    /// États possibles de l'abeille.
    /// </summary>
    public enum State
    {
        Patrouille,
        FlyUp,
        Dive,
        Stunt,
        InWater,
    }

    /// <summary>
    /// Phases utilisées pendant le déplacement aérien avant l'attaque.
    /// </summary>
    enum FlyPhase
    {
        AllerAuCentre,
        AllerAGauche,
        AllerADroite,
        Fini
    }

    /// <summary>
    /// Phase actuelle du déplacement aérien.
    /// </summary>
    private FlyPhase _flyPhase = FlyPhase.AllerAuCentre;

    /// <summary>
    /// État actuel de l'abeille.
    /// </summary>
```

```
public State currentState;

/// <summary>
/// Rigidbody2D utilisé pour gérer les déplacements.
/// </summary>
private Rigidbody2D _rb;

/// <summary>
/// Animator utilisé pour gérer les animations.
/// </summary>
private Animator _anim;

/// <summary>
/// SpriteRenderer utilisé pour retourner le sprite.
/// </summary>
private SpriteRenderer _sr;

/// <summary>
/// Vitesse actuelle de l'abeille.
/// </summary>
private float _speed;

/// <summary>
/// Vitesse utilisée pendant la patrouille.
/// </summary>
private float _patrolSpeed = 2f;

/// <summary>
/// Vitesse utilisée pendant la montée avant l'attaque.
/// </summary>
private float _flyUpSpeed = 4f;

/// <summary>
/// Vitesse utilisée pendant la plongée.
/// </summary>
private float _diveSpeed = 10f;

/// <summary>
/// Durée pendant laquelle l'abeille est étourdie.
/// </summary>
private float _stunDuration = 4f;

/// <summary>
/// Timer de l'état étourdi.
/// </summary>
private float _stunTimer = 0f;

/// <summary>
/// Direction de déplacement de l'abeille.
/// </summary>
private float _direction = 1;

/// <summary>
/// Détecteur d'obstacle à droite.
```

```
/// </summary>
public GameObject DetecteurDroit;

/// <summary>
/// Détecteur d'obstacle à gauche.
/// </summary>
public GameObject DetecteurGauche;

/// <summary>
/// Détecteur du joueur et du sol.
/// </summary>
public GameObject DetecteurPlayerEtSol;

/// <summary>
/// Script du détecteur d'obstacle à droite.
/// </summary>
DetecterObstacle _detecterObstacleDroit;

/// <summary>
/// Script du détecteur d'obstacle à gauche.
/// </summary>
DetecterObstacle _detecterObstacleGauche;

/// <summary>
/// Script permettant de détecter le joueur.
/// </summary>
DetecterEnnemiSol _detecterEnnemiSol;

/// <summary>
/// Référence au sol détecté.
/// </summary>
GameObject _Sol;

/// <summary>
/// Référence au joueur détecté.
/// </summary>
GameObject _player;

/// <summary>
/// Distance entre l'abeille et le sol.
/// </summary>
float _distanceEntreAbeilleEtSol;

/// <summary>
/// Distance maximale entre l'abeille et le sol.
/// </summary>
float _distanceMaxEntreAbeilleEtSol = 5.5f;

/// <summary>
/// Distance minimale entre l'abeille et le sol.
/// </summary>
float _distanceMinEntreAbeilleEtSol = 4.5f;

/// <summary>
```

```
/// Distance actuelle entre l'abeille et le sol.
/// </summary>
float _distanceActuelleEntreAbeilleEtSol;

//FlyUp

//Calcul Distance Y

/// <summary>
/// Distance actuelle entre l'abeille et le joueur.
/// </summary>
float _distanceActuelleEntreAbeilleEtPlayer;

/// <summary>
/// Distance maximale entre l'abeille et le joueur.
/// </summary>
float _distanceMaxEntreAbeilleEtPlayer = 5.5f;

/// <summary>
/// Distance minimale entre l'abeille et le joueur.
/// </summary>
float _distanceMinEntreAbeilleEtPlayer = 4.5f;

//Calcul Distance X

/// <summary>
/// Vérifie si l'abeille est actuellement en train d'attaquer.
/// </summary>
public bool IsAttacking = false;

/// <summary>
/// Timer de la phase FlyUp.
/// </summary>
float _flyUpTimer = 0f;

/// <summary>
/// Temps maximum de la phase FlyUp.
/// </summary>
float _maxFlyUpTime = 3.5f;

//position du joueur au moment du dive;

/// <summary>
/// Position du joueur ciblée au moment de la plongée.
/// </summary>
Transform _targetPlayerPosition;

/// <summary>
/// Entité touchée par l'abeille.
/// </summary>
Entity _entity;

/// <summary>
/// Direction de déplacement de l'abeille pendant la plongée.
```



```

    /// </summary>
    Vector2 _directionBee;

    /// <summary>
    /// Limite droite de la zone de patrouille.
    /// </summary>
    public GameObject LimitePatrouilleDroite;

    /// <summary>
    /// Limite gauche de la zone de patrouille.
    /// </summary>
    public GameObject LimitePatrouilleGauche;

    /// <summary>
    /// Initialisation des composants nécessaires.
    /// </summary>
    void Start()
    {
        IsDead = false;
        _rb = GetComponent<Rigidbody2D>();
        _anim = GetComponentInChildren<Animator>();
        _sr = GetComponentInChildren<SpriteRenderer>();
        _detecterObstacleDroit = DetecteurDroit.GetComponent<DetecterObstacle>();
        _detecterObstacleGauche = DetecteurGauche.GetComponent<DetecterObstacle>
    );
        currentState = State.Patrouille;
        _detecterEnnemiSol = DetecteurPlayerEtSol.GetComponent<DetecterEnnemiSol>
    );
        _speed = _patrolSpeed;
    }

    /// <summary>
    /// Mise à jour appelée à chaque frame.
    /// Gère l'état actuel de l'abeille.
    /// </summary>
    void Update()
    {
        //Debug.Log(currentState);
        if (Hurt == true)
        {
            _anim.SetTrigger("Hurt");
            Hurt = false;
        }
        else
        {
            flip();
            if(IsInWater)
            {
                currentState = State.InWater;
            }
            //Si pas de joueur détecter et qu'il n'est pas entrain d'attaquer

```

```
        if (_detectorEnnemiSol.player != null && !IsAttacking)
        {
            _player = _detectorEnnemiSol.player;
            currentState = State.FlyUp;
        }
        switch (currentState)
        {
            case State.Patrouille:
                // Logique de patrouille
                Patrouille();
                //ResetAttack();
                break;
            case State.FlyUp:
                FlyUp();
                // Logique de vol vers le haut
                break;
            case State.Dive:
                Dive();
                // Logique de plongée
                break;
            case State.Stunt:
                Stunt();
                // Logique de stun
                break;
            case State.InWater:
                InWater();
                // Logique de stun
                break;
        }
    }

}

/// <summary>
/// Gère le comportement de l'abeille lorsqu'elle est dans l'eau.
/// </summary>
void InWater()
{
    _rb.gravityScale = 2f;
    _rb.linearVelocity = new Vector2(0, 0);
    Invoke("PrendreDesDegats", 2f);
}

/// <summary>
/// Inflige des dégâts à l'abeille.
/// </summary>
void PrendreDesDegats()
{
    TakeDamage(1);
}

/// <summary>
/// Retourne le sprite selon la direction de déplacement.
/// </summary>
```

```

void flip()
{
    if (_direction == 1)
    {
        _sr.flipX = true;
    }
    else
    {
        _sr.flipX = false;
    }
}

/// <summary>
/// Gère le déplacement de patrouille de l'abeille.
/// </summary>
void Patrouille()
{
    _speed = _patrolSpeed;
    _rb.linearVelocity = new Vector2(_direction * _speed,
    _rb.linearVelocity.y);

    if(transform.position.x > LimitePatrouilleDroite.transform.position.x
|| _detecterObstacleDroit.isObstacleDetected)
    {
        _direction = -1;
    }
    else if(transform.position.x < LimitePatrouilleGauche.transform.position.x
|| _detecterObstacleGauche.isObstacleDetected)
    {
        _direction = 1;
    }

    if(transform.position.y < LimitePatrouilleGauche.transform.position.y -
0.4f)
    {
        _rb.linearVelocity = new Vector2(_rb.linearVelocity.x, _speed);
    }
    if(transform.position.y > LimitePatrouilleDroite.transform.position.y +
0.4f)
    {
        _rb.linearVelocity = new Vector2(_rb.linearVelocity.x, -_speed);
    }

}

//contient du code générer par l'ia
/// <summary>
/// Gère la phase où l'abeille monte et se place autour du joueur avant de
plonger.
/// </summary>
void FlyUp()
{
    IsAttacking = true;

```

```
if (_player == null)
{
    currentState = State.Patrouille;
    return;
}

float velocityX = 0f;
float velocityY = 0f;

_speed = _flyUpSpeed;

float distanceY = transform.position.y - _player.transform.position.y;

if (distanceY < _distanceMinEntreAbeilleEtPlayer)
{
    velocityY = _speed;
}
else if (distanceY > _distanceMaxEntreAbeilleEtPlayer)
{
    velocityY = -_speed;
}
else
{
    velocityY = 0f;
}

float positionCibleX = _player.transform.position.x;
float pointGauche = positionCibleX - 2.5f;
float pointDroit = positionCibleX + 2.5f;

float marge = 0.1f;

switch (_flyPhase)
{
    case FlyPhase.AllerAuCentre:
        if (transform.position.x < positionCibleX - marge)
        {
            velocityX = _speed;
        }
        else if (transform.position.x > positionCibleX + marge)
        {
            velocityX = -_speed;
        }
        else
        {
            _flyPhase = FlyPhase.AllerAGauche;
        }
        break;

    case FlyPhase.AllerAGauche:
        if (transform.position.x > pointGauche + marge)
        {

```

```

        velocityX = -_speed;
    }
    else
    {
        _flyPhase = FlyPhase.AllerADroite;
    }
    break;

case FlyPhase.AllerADroite:
    if (transform.position.x < pointDroit - marge)
    {
        velocityX = _speed;
    }
    else
    {
        _flyPhase = FlyPhase.AllerAGauche;
    }
    break;

case FlyPhase.Fini:
    //Debug.Log("Fin de la phase de vol vers le haut");
    currentState = State.Dive;
    _flyPhase = FlyPhase.AllerAuCentre;
    break;
}

_rb.linearVelocity = new Vector2(velocityX, velocityY);
_flyUpTimer += Time.deltaTime;
if (_flyUpTimer >= _maxFlyUpTime)
{
    _flyUpTimer = 0f;
    _flyPhase = FlyPhase.Fini;
}
}

/// <summary>
/// Gère la plongée de l'abeille vers la position du joueur.
/// </summary>
void Dive()
{
    if (_targetPlayerPosition == null)
    {
        _targetPlayerPosition = _player.transform;

        _directionBee = (_targetPlayerPosition.position -
transform.position).normalized;

        return;
    }
    _rb.linearVelocity = _directionBee * _diveSpeed;
}

```

```
/// <summary>
/// Gère l'état étourdi de l'abeille après une collision.
/// </summary>
void Stunt()
{
    _stunTimer += Time.deltaTime;
    _rb.linearVelocity = Vector2.zero;
    if (_stunTimer >= _stunDuration)
    {
        _stunTimer = 0f;
        currentState = State.Patrouille;
        IsAttacking = false;
        _targetPlayerPosition = null;
    }
}

/// <summary>
/// Fonction appelée lorsqu'une collision commence.
/// Sert à gérer les dégâts au joueur ou le stun de l'abeille.
/// </summary>
/// <param name="collision">
/// Collision détectée.
/// </param>
void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.CompareTag("Player") && currentState == State.Dive)
    {
        {
            _player = collision.gameObject;
            if (_player.name == "Ours")
            {
                _entity = _player.GetComponentInParent<Bear>();
            }
            if (_player.name == "Druide")
            {
                _entity = _player.GetComponentInParent<Player>();
            }
            if (_player.name == "Poisson")
            {
                _entity = _player.GetComponentInParent<Fish>();
            }
            if (_player.name == "Oiseau")
            {
                _entity = _player.GetComponentInParent<Bird>();
            }

            if (_entity != null && _entity.Hurt == false)
            {
                //Debug.Log("Player touché par l'abeille");
            }
        }
    }
}
```

```
        EnleverDegat();

    }
}
else if (currentState == State.Dive)
{
    currentState = State.Stunt;
    _stunTimer = 0f;
}
}

/// <summary>
/// Fonction appelée lorsqu'une collision continue.
/// Réinitialise l'attaque si l'abeille reste bloquée pendant une plongée.
/// </summary>
/// <param name="collision">
/// Collision détectée.
/// </param>
void OnCollisionStay2D(Collision2D collision)
{
    if(currentState == State.Dive)
    {
        currentState = State.Patrouille;
        Invoke("ResetAttack",1f);
        _stunTimer = 0f;
    }

}

/// <summary>
/// Inflige des dégâts au joueur détecté.
/// </summary>
void EnleverDegat()
{
    if (_entity != null)
    {
        _anim.SetTrigger("Attack");
        _entity.TakeDamage(1);
        _entity = null;
        currentState = State.Patrouille;
        IsAttacking = false;
        _targetPlayerPosition = null;
    }
}

/// <summary>
/// Réinitialise l'état d'attaque de l'abeille.
/// </summary>
void ResetAttack()
{
    IsAttacking = false;
}
}
```

DetectorEnnemiSol

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Legacy-Fantasy/Mob/Small
Bee/Script/DetectorEnnemiSol.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé finalement pour détecter seulement le joueur
using UnityEngine;

public class DetectorEnnemiSol : MonoBehaviour
{
    // Start is called once before the first execution of Update after the
    MonoBehaviour is created
    /// <summary>
    /// Le GameObject du joueur
    /// </summary>
    public GameObject player;
    /// <summary>
    /// bool pour savoir si le player est dans la zone
    /// </summary>
    bool playerIn;

    /// <summary>
    /// Fonction appelée tant qu'un collider entre dans le trigger
    /// </summary>
    /// <param name="collision">Quand le collider détecte une collision</param>
    void OnTriggerEnter2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            player = collision.gameObject;
            playerIn = true;
        }
    }

    /// <summary>
    /// Fonction appelée tant qu'un collider reste
    /// dans la zone de détection.
    /// </summary>
    /// <param name="collision">Quand le collider détecte une collision</param>
    void OnTriggerStay2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            player = collision.gameObject;
            playerIn = true;
        }
    }
}
```



```

    /// <summary>
    /// Fonction appelée quand un collider sort
    /// de la zone de détection.
    /// </summary>
    /// <param name="collision">Quand le collider détecte plus la
collision</param>
    void OnTriggerExit2D(Collider2D collision)
    {
        if (collision.gameObject.CompareTag("Player"))
        {
            player = null;
            playerIn = false;
        }
    }
}

```

DetectorObstacle

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Legacy-Fantasy/Mob/Small Bee/Script/DetectorObstacle.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour détecter les obstacles de l'abeille
using UnityEngine;

public class DetectorObstacle : MonoBehaviour
{
    /// <summary>
    /// Vérifie si un obstacle est actuellement détecté.
    /// </summary>
    public bool isObstacleDetected;

    /// <summary>
    /// Initialisation des variables.
    /// </summary>
    void Start()
    {
        isObstacleDetected = false;
    }

    /// <summary>
    /// Fonction appelée automatiquement lorsqu'un collider
    /// entre dans la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider détecté par le trigger.

```

```

    /// </param>
    void OnTriggerEnter2D(Collider2D collision)
    {
        // Vérifie que l'objet détecté n'est pas le joueur
        if (!collision.gameObject.CompareTag("Player"))
        {
            // Un obstacle est détecté
            isObstacleDetected = true;
        }
    }

    /// <summary>
    /// Fonction appelée automatiquement lorsqu'un collider
    /// quitte la zone de détection.
    /// </summary>
    /// <param name="collision">
    /// Collider qui quitte le trigger.
    /// </param>
    void OnTriggerExit2D(Collider2D collision)
    {
        // Aucun obstacle détecté
        isObstacleDetected = false;
    }
}

```

DepotCrystal

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Portail/Script/DepotCrystal.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour déposer le cristal
using UnityEngine;

public class DepotCrystal : MonoBehaviour
{
    /// <summary>
    /// Vérifie si le dépôt de cristal est activé.
    /// </summary>
    public bool isActivated = false;

    /// <summary>
    /// Animator utilisé pour jouer l'animation du dépôt.
    /// </summary>
    Animator _anim;

    /// <summary>
    /// Initialisation des composants nécessaires.
    /// </summary>

```

```
void Start()
{
    _anim = GetComponentInChildren<Animator>();

    // Désactive l'animation au démarrage
    _anim.enabled = false;
}

/// <summary>
/// Mise à jour appelée à chaque frame.
/// Vérifie si un cristal est présent dans le dépôt.
/// </summary>
void Update()
{
    // Recherche le cristal dans les enfants du dépôt
    Transform crystal = transform.Find("crystal");

    // Vérifie si un cristal est présent
    if(crystal != null)
    {
        // Replace le cristal à la position prévue
        crystal.position =
            transform.Find("positionCrystal").position;

        // Active l'animation du dépôt
        _anim.enabled = true;

        // Active le rendu visuel du cristal
        crystal
            .gameObject
            .GetComponentInChildren<SpriteRenderer>()
            .enabled = true;

        // Active le dépôt
        isActivated = true;
    }
}
```

FinirLeNiveau

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Portail/Script/FinirLeNiveau.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour terminer le niveau
using System;
using NUnit.Framework;
using Unity.VisualScripting;
```

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class FinirLeNiveau : MonoBehaviour
{
    /// <summary>
    /// Référence au cristal porté par le joueur.
    /// </summary>
    Transform _crystal;

    /// <summary>
    /// Vérifie si le joueur possède le cristal nécessaire pour finir le niveau.
    /// </summary>
    public bool canBeTake;

    /// <summary>
    /// Vérifie si le niveau est terminé.
    /// </summary>
    bool _isFinish;

    /// <summary>
    /// Nom de la scène du prochain niveau.
    /// </summary>
    public string NomSceneProchaineNiveau;

    /// <summary>
    /// Initialisation des variables.
    /// </summary>
    void Start()
    {
        _isFinish = false;
        canBeTake = false;
    }

    /// <summary>
    /// Retourne l'état de fin du niveau.
    /// </summary>
    /// <returns>
    /// True si le niveau est terminé, sinon false.
    /// </returns>
    public bool IsFinished()
    {
        return _isFinish;
    }

    // Update is called once per frame

    /// <summary>
    /// Fonction appelée lorsqu'une collision commence.
    /// Vérifie si le joueur possède le cristal pour terminer le niveau.
    /// </summary>
    /// <param name="collision">
    /// Collision détectée.
    /// </param>
```

```
void OnCollisionEnter2D(Collision2D collision)
{
    Debug.Log("Collision détectée avec : " + collision.gameObject);

    // Vérifie si l'objet en collision est le joueur
    if (collision.gameObject.CompareTag("Player"))
    {
        // Parcourt les enfants du joueur pour trouver le cristal
        foreach (Transform child in
collision.gameObject.GetComponentsInChildren<Transform>())
        {
            if (child.name == "crystal")
            {
                _crystal = child;
                break;
            }
        }

        // Vérifie si le cristal a été trouvé
        if (_crystal != null)
        {
            Debug.Log("crystal trouver");
            canBeTake = true;
        }
        else
        {
            Debug.Log("Crystal non trouvé !");
        }

        // Si le joueur possède le cristal, le niveau est terminé
        if (canBeTake)
        {
            GestionPersonnage gp =
                collision.gameObject.GetComponentInParent<GestionPersonnage>
());

            // Sauvegarde les statistiques du joueur
            PlayerStatManager.Instance.SauvegarderDonnee(
                gp.MaxHealth,
                gp.RetournerVie(),
                gp.MaxMana,
                gp.RetournerMana()
            );

            _isFinish = true;
        }
    }
}
```

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Trap/Brick/Script/Brick.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour gérer les brick
using System;
using System.Collections.Generic;
using Unity.VisualScripting;
using UnityEngine;

public class Brick : Entity
{
    // Start is called once before
    // the first execution of Update after the MonoBehaviour is created

    /// <summary>
    /// Liste contenant les GameObjects des sprites de la brique.
    /// </summary>
    private List<GameObject> _bricksSprites;

    /// <summary>
    /// Tableau contenant tous les Transform enfants de la brique.
    /// </summary>
    private Transform[] _bricksSpritesTransforms;

    /// <summary>
    /// Initialisation des sprites de la brique.
    /// </summary>
    void Start()
    {
        _bricksSpritesTransforms = gameObject.GetComponentsInChildren<Transform>
        (
        );
        _bricksSprites = new List<GameObject>();

        foreach (Transform item in _bricksSpritesTransforms)
        {
            _bricksSprites.Add(item.gameObject);

            if(item.gameObject != gameObject) // Vérifie que ce n'est pas le
parent
            {
                item.GetComponent<Animator>().enabled = false; // Désactive les
animations au départ
            }
        }
    }

    /// <summary>
    /// Mise à jour appelée à chaque frame.
    /// Vérifie si la brique est détruite.
    /// </summary>
```

```

    void Update()
    {
        if(IsDead)
        {
            DestructionBrick();
        }
    }

    /// <summary>
    /// Active les animations de destruction
    /// des différents morceaux de la brique.
    /// </summary>
    void DestructionBrick()
    {
        foreach (var item in _bricksSprites)
        {
            if(item.gameObject != gameObject) // Vérifie que ce n'est pas le
parent
                {
                    item.GetComponent<Animator>().enabled = true; // Active les
animations de destruction
                }
        }

        // Détruit les morceaux après un court délai
        Invoke("DestroyBrick", 0.5f);
    }

    /// <summary>
    /// Détruit tous les sprites de la brique.
    /// </summary>
    void DestroyBrick()
    {
        foreach (var item in _bricksSprites)
        {
            Destroy(item); // Détruit les sprites de la brique
        }
    }
}

```

SpikeAnimationState

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Trap/spikeGround/Script/SpikeAnimationState.cs

```

//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour activer les dégats pendant à une animation
using UnityEngine;
using System.Collections;

```

```
public class SpikeAnimationState : MonoBehaviour
{
    /// <summary>
    /// Vérifie si les pics sont actuellement en attaque.
    /// </summary>
    public bool attack = false;

    /// <summary>
    /// Animator utilisé pour gérer les animations des pics.
    /// </summary>
    Animator _animator;

    /// <summary>
    /// Référence au script principal des pics.
    /// </summary>
    spikeGroundScript _spikeGroundScript;

    /// <summary>
    /// Collider utilisé pour infliger des dégâts.
    /// </summary>
    BoxCollider2D _boxCollider2D;

    /// <summary>
    /// Initialisation des composants nécessaires.
    /// </summary>
    void Start()
    {
        _animator = GetComponent<Animator>();

        // Lance la boucle d'attaque des pics
        StartCoroutine(SpikeLoop());

        _spikeGroundScript =
            GetComponentInParent<spikeGroundScript>();

        _boxCollider2D =
            GetComponentInParent<BoxCollider2D>();
    }

    /// <summary>
    /// Coroutine permettant de répéter l'attaque des pics.
    /// </summary>
    IEnumerator SpikeLoop()
    {
        while (true)
        {
            // Lance l'animation d'attaque
            _animator.SetTrigger("Attack");

            // Attend 2 secondes avant la prochaine attaque
            yield return new WaitForSeconds(2f);
        }
    }
}
```



```
/// <summary>
/// Active le collider des pics.
/// Fonction appelée via un Animation Event.
/// </summary>
public void AttaqueTrue()
{
    _boxCollider2D.enabled = true;
}

/// <summary>
/// Désactive le collider des pics
/// et réinitialise l'animation.
/// Fonction appelée via un Animation Event.
/// </summary>
public void AttaqueFalse()
{
    // Stop temporairement l'Animator
    _animator.enabled = false;

    // Désactive le collider
    _boxCollider2D.enabled = false;

    attack = false;

    // Réactive l'Animator
    _animator.enabled = true;
}

/// <summary>
/// Réinitialise l'état de dégâts des pics.
/// Fonction appelée via un Animation Event.
/// </summary>
public void ResetHurt()
{
    _spikeGroundScript.ResetHurt();
}
}
```

spikeGroundScript

Chemin : C:/Users/Admin/Druid's Adventure/Assets/Asset/Trap/spikeGround/Script/spikeGroundScript.cs

```
//Valentin Tercier
//07.05.2026
//Ce script est utilisé pour gérer le piège
using UnityEngine;

public class spikeGroundScript : MonoBehaviour
```

```
{
    /// <summary>
    /// Vérifie si le joueur est dans la zone des pics.
    /// </summary>
    bool _playerIn;

    /// <summary>
    /// Référence au script gérant l'animation des pics.
    /// </summary>
    private SpikeAnimationState _spikeAnimationState;

    /// <summary>
    /// Vérifie si le joueur vient de recevoir des dégâts.
    /// </summary>
    bool _hurt = false;

    /// <summary>
    /// Entité du joueur touché par les pics.
    /// </summary>
    Entity _entity;

    /// <summary>
    /// GameObject du joueur touché par les pics.
    /// </summary>
    private GameObject _player;

    /// <summary>
    /// Initialisation des composants nécessaires.
    /// </summary>
    void Start()
    {
        _spikeAnimationState = GetComponentInChildren<SpikeAnimationState>();
    }

    // Update is called once per frame

    /// <summary>
    /// Inflige des dégâts au joueur touché
    /// et le repousse légèrement.
    /// </summary>
    public void EnleverDegat()
    {
        if(_entity != null)
        {
            _entity.TakeDamage(1);
            _player.GetComponent<Rigidbody2D>().AddForce(new Vector2(50, 5),
ForceMode2D.Impulse);
            _hurt = true;
        }
        else
        {
            Debug.Log("Player est null");
        }
    }
}
```

```
/// <summary>
/// Met l'état de dégâts à true.
/// </summary>
void Hurt()
{
    _hurt = true;
}

/// <summary>
/// Réinitialise l'état de dégâts.
/// </summary>
public void ResetHurt()
{
    _hurt = false;
}

/// <summary>
/// Fonction appelée lorsqu'une collision commence avec les pics.
/// </summary>
/// <param name="collision">
/// Collision détectée.
/// </param>
void OnCollisionEnter2D(Collision2D collision)
{
    Debug.Log("OnTriggerEnter2D" + collision.gameObject.name);

    _player = collision.gameObject;

    // Vérifie si l'objet touché est l'ours
    if(_player.name == "Ours")
    {
        _entity = _player.GetComponentInParent<Bear>();
    }

    // Vérifie si l'objet touché est le druide
    if(_player.name == "Druide")
    {
        _entity = _player.GetComponentInParent<Player>();
    }

    // Vérifie si l'objet touché est le poisson
    if(_player.name == "Poisson")
    {
        _entity = _player.GetComponentInParent<Fish>();
    }

    // Vérifie si l'objet touché est l'oiseau
    if(_player.name == "Oiseau")
    {
        _entity = _player.GetComponentInParent<Bird>();
    }

    // Vérifie si une entité valide est détectée
```

```
        if (_entity != null && _entity.Hurt == false)
        {
            Debug.Log("Player touché par les piques");
            EnleverDegat();
        }
    }

    /// <summary>
    /// Fonction appelée lorsqu'une collision se termine avec les pics.
    /// </summary>
    /// <param name="collision">
    /// Collision qui se termine.
    /// </param>
    void OnCollisionExit2D(Collision2D collision)
    {
        Debug.Log("OnTriggerExit2D" + collision.gameObject.name);

        _player = null;
        _entity = null;
    }
}
```
